Lecture 6

Propositional Logic 2

Intelligent Systems
Vrije Universiteit

Overview

1. SAT Solving
2. Clause Normal Form (CNF)
3. Davis-Putnam Algorithm (DPLL)
4. Entailment by Refutation with DPLL

Part 1: Satisfiability and SAT Solving

Satisfiability, Validity, Entailment

A sentence is satisfiable if it is true in some interpretation, i.e. there exists a model.

e.g. A ∨ B, C

A sentence is unsatisfiable if it is true in no interpretation, i.e. it has no model.

e.g. A ⋀ ¬A

A sentence is valid if it is true in all interpretations, also called a tautology.

e.g. A ∨ ¬A

A sentence is entailed by a KB K *iff* is true in all models of KB. We write KB ⊨ *a*.

e.g. {¬A, A ∨ B} ⊨ B

Let's look at more useful definitions for describing knowledge bases and their models. If we can find a model for a sentence, then it is **satisfiable.** Otherwise, it is **unsatisfiable** because there does not exist a model.
A sentence is **valid** if it is always true, meaning whichever truth values we come up with the sentence will *always* be true.
When looking at a knowledge base and its models, a sentence which is true in all the models is considered **entailed**. In the example provided, B must be true in order for a model to be true, so it is entailed.

SAT Solving

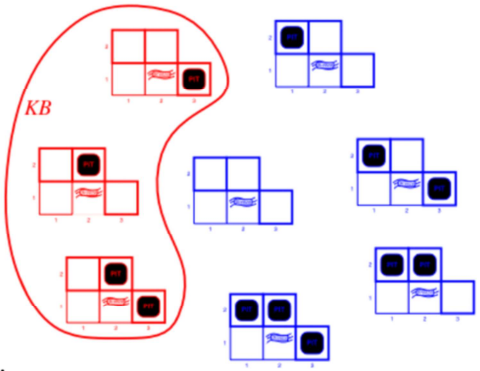
Satisfiability solving is the task of finding one or more models of the KB.

It is an old but still active field of research, used in hardware/software verification, planning & scheduling, control, protocol design, etc.

Example: In game scenarios th KB represents rules and game states. The models correspond to valid states in the game.

The red states correspond to the models of Wumpus World.

KB = wumpus-world-rules + observations



An example of SAT solving is demonstrated using Wumpus World. In all the different states shown on the screen, only the states which agree with the observations and rules of the game are models.

Knowledge Bases & Models *Recap*

Recall that a knowledge base is a set of true sentences.

e.g. {¬A, A ∨ B, C}

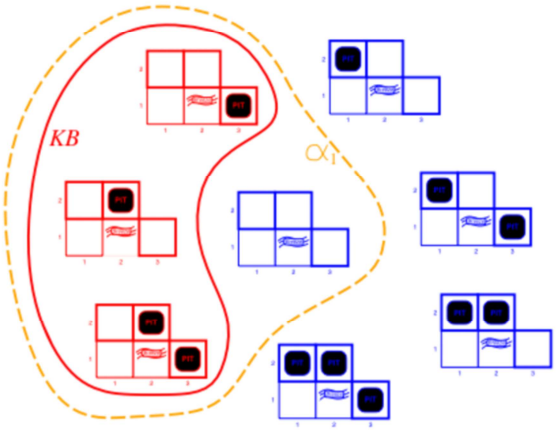
Models are interpretation functions which evaluate all formulas in a KB to true.

e.g. v(A) = 0, v(B) = 1, v(C) = 1 is model for {¬A, A ∨ B, C}

A sentence is satisfiable if there exists *at least one model*: SAT Solving is checking whether there is one.

A sentence *a* is entailed by a KB *k* iff *a* is true in all models of KB. KB ⊨ B

Wumpus World *Recap*

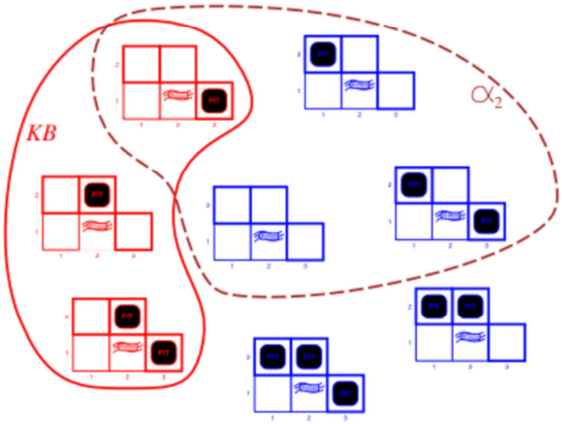


KB = wumpus-world rules + observations

α_1 = "[1,2] is safe", $KB \models \alpha_1$, proved by model checking

Note: these interpretations only deal with the variable for Pits, not Breezes. Otherwise, there should be Breezes in the models of KB in 1.2.

Wumpus World *Recap*



KB = wumpus-world rules + observations

α_2 = "[2,2] is safe", $KB \not\models \alpha_2$

Entailment as Satisfiability

We can check what follows from a knowledge base by checking unsatisfiability, or nonexistence of a model.

Example

Is $P_{1,2}$ entailed by the KB? = Is $P_{1,2}$ true in all models of KB?

This is the same as asking whether KB $\wedge \neg P_{1,2}$ is unsatisfiable. If it is, then we know that KB entails $P_{1,2}$.

This statement is formulated in such a way that it is true whenever the KB is formulate as a single PL formula. If the KB is formulated as a set of formulas, the statement has to be KB U $\{\neg P_{1,2}\}$ is unsatisfiable to be formally correct.

Part 2: Calculus: SAT solving and Entailment

Intelligent Systems
Vrije Universiteit

In order to solve the problem of checking satisfiability with truth tables, we can write our sentences in a different notation to make this process easier. This notation is **clause normal form.**

Checking Satisfiability with Truth Tables *Example*

Check satisfiability of the sentence **S = (P ∨ ¬Q) ∧ (Q ∨ R) ∧ (¬R ∨ ¬P)**

P	Q	R	P v -Q	Q v R	-R v –P	S
True	True	True	True	True	False	False
True	True	False				
True	False	True				
True	False	False				
False	True	True				
False	True	False				
False	False	True				
False	False	False				

Problem!
We need 2^n solutions.
3 variables: 8 variables
20 variables: 1048576
50 variables:
1125899906842624
…

Part 2:.1 Clause Normal Form (CNF)

In order to solve the problem of checking satisfiability with truth tables, we can write our sentences in a different notation to make this process easier. This notation is **clause normal form.**

Clause Normal Form (CNF)

A literal is a (negated) proposition, e.g. "*P*", "¬*P*", "Q" are three literals

Clause/Conjunctive Normal Form is a notation in which there are only conjunctions of disjunctions of literals. Recall: conjunction ∧ and disjunction ∨

Examples of CNF

(Q ∨ R) ∧ (¬R ∨ ¬P) where *Q, R,* ¬*R,* and ¬*P* are literals

(R ∨ ¬P) where *R* and ¬*P* are literals

Counter Example of not CNF

(P ∧ ¬Q) ∨ (Q ∧ R) or P ∧ ¬(¬R ∨ ¬P)

Note: during Lecture 6 2023 there were mistakes on this slide that have been fixed in this version

Calculating CNF *Steps*

1. Remove equivalences

   $P \Leftrightarrow Q$ rewrites to $(P \rightarrow Q) \wedge (Q \rightarrow P)$

1. Remove implications

   $P \rightarrow Q$ rewrites to $(\neg P \vee Q)$

1. Move negations inward (de Morgan Rule)

   $\neg(P \vee Q)$ rewrites to $\neg P \vee \neg Q$

   $\neg(P \wedge Q)$ rewrites to $\neg P \vee \neg Q$

1. Move conjunctions outward (distribution)

   $R \vee (P \wedge Q)$ rewrites to $(R \vee P) \wedge (R \vee Q)$

1. Split up conjunctive clauses

   $(P \vee Q) \wedge (Q \vee R)$ rewrites to $(P \vee Q), (Q \vee R)$

Applying rules in this order *always* produces CNF.

Calculating CNF *Example*

$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$

- Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow$ ß with $(\alpha \Rightarrow$ ß$) \wedge ($ß $\Rightarrow \alpha)$.
  $(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$
- Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow$ ß with $\neg\, \alpha \vee$ ß.
  $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$
- Move $\neg$ inwards using de Morgan's rules and double-negation:
  $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$
- Apply distributivity law ($\wedge$ over $\vee$) and flatten:
  $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$

CNF as Sets

A CNF formula

$(P \vee Q) \wedge (Q \vee R)$

Can be written as a list of lists. "∧" becomes ","

$\{\{P \vee Q\}, \{Q \vee R\}\}$

Why? → Easy and efficient to implement.

Exercise Transform $(A \Leftrightarrow B) \wedge C$ into CNF.

16

CNF notation can be written in terms of sets by replacing the conjunction symbols with commas. This is because in CNF we assume that *every* clause has to be true in order for the full sentence to be true. This will be useful for the next section.

The answer to the exercise in the notes for the next slide.

Part 2,2: Davis-Putnam Algorithm (DPLL)

CNF is useful because we can use it as input for an algorithm, such as Davis-Putnam or DPLL.

Answer to previous exercise: $((\neg A \vee B) \wedge (\neg B \vee A) \wedge C)$ or $\{\{\neg A \vee B\}, \{\neg B \vee A\}, \{C\}\}$

DPLL Background

- Davis-Putnam-Logemann-Loveland algorithm (DPLL) is a refinement of the original algorithm by Davis & Putnam (DP)
- First introduced in the 1960s
- Original DP used potentially exponential space, improved over the years
- We need CNF to start DPLL!

DPLL Algorithm

DPLL is an algorithm for deciding satisfiability of propositional formulas.

Method

1. Start from CNF formulas
2. Assign truth values (partial assignments) to literals, which are single symbols like "*p*", "q", of a sentence one-by-one using *backtracking.*

Goal

Find a full assignment (all literals have a truth value) that satisfies the sentence *S*.

DPLL is exponential in the worst case but complete.

DP *Version 1 Davis-Putnam*

Let *pa* be a partial assignment.

```
dpll_1(S,pa){
    if (pa makes S false) return false;
    if (pa makes S true) return true;

    choose P in S;
        if (dpll_1(pa U {P=False}))
            return true;
        else
                return dpll_1(pa U {P=True}); }
```

DPLL returns true if *S* is satisfiable, false otherwise.

We will work through three versions of DPLL, each one improving on some problem.

When starting, the DPLL algorithm has no assignments for any propositions. It starts by making a partial assignment, such as 'P is true.' If this partial assignment makes the whole sentence false, we return false, otherwise we return true. An important concept to remember is that the algorithm returns true if the sentence is satisfiable, and false otherwise.

Let's look at an example.

DPLL *Version 1*

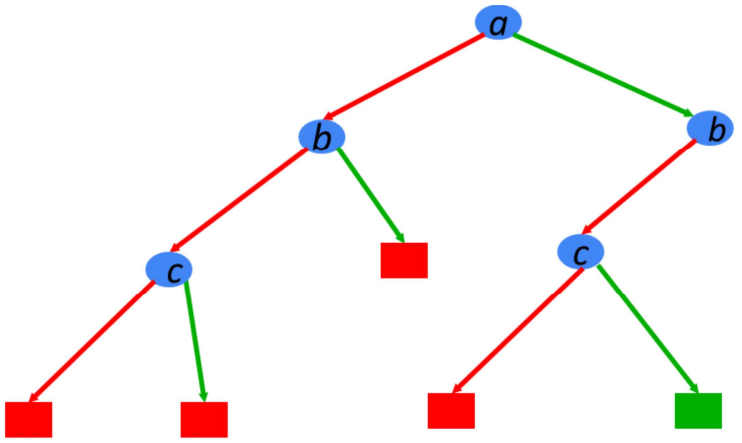
Clauses

$(a \vee b \vee c)$

$(a \vee \neg b)$

$(a \vee \neg c)$

$(\neg a \vee c)$



DPLL *as Search*

Nodes in the search tree correspond to partial assignments as sets of chosen literals on the path from the root to the node.

Goal states are full assignments that satisfy *S*.

- There may be more than one answer or full assignments
- These states correspond to models

DPLL uses *depth-first search.*

DPLL *Formalisation*

If literal L_1 is true, then clause (L_1 v L_2 v …) is true.

If clause C_1 is true, then ($C_1 \wedge C_2 \wedge C_3 \wedge \ldots$) has the same value as ($C_2 \wedge C_3 \wedge \ldots$).

Therefore it is okay to delete clauses containing true literals

If literal L_1 is false, then clause (L_1 v L_2 v L_3v…) has the same value as clause (L_2 v L_3v…)

Therefore it is okay to delete false literals from clauses

If literal L_1 is false, then clause (L_1) is false.

If a clause is empty, it is false

This slide has been adapted after the lecture 6 2023, because it contained a number of mistakes.

DPLL *Version 2*

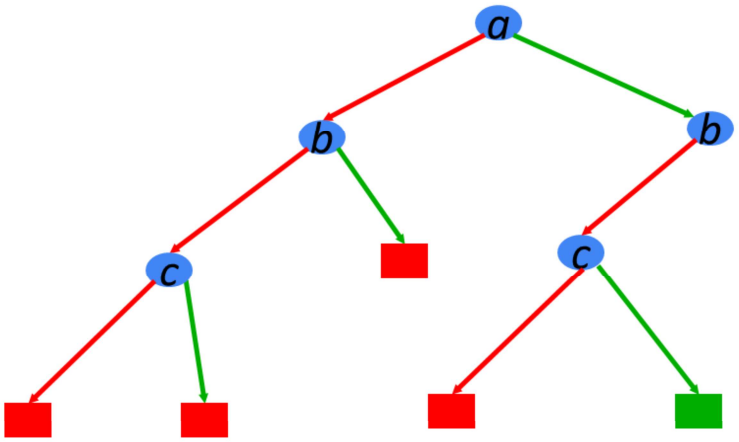
Let S be a set of clauses.

```
dpll_2(S, literal){
   remove clauses containing literal
   shorten clauses containing ¬literal

   if (S contains no clauses)
                return true;
   if (S contains empty clause)
                return false;
   choose P in S;
        if (dpll_2(S, ¬P))
                return true;
        else
                return dpll_2(S, P);
}
```

DPLL *Version 2*

(☐ ☐ c)
(☐ $\neg b$)
(☐ $\neg c$)
(☐ c)



DPLL *Further Improvements*

A unit clause is a clause that contains a single literal L.
L must be true to satisfy the clause set.

Let L be a Literal. Then the sentence ($L \wedge C_2 \wedge C_3 \ldots$) is only true when L is true.

Therefore branch immediately on unit literals

If literal L does not appear negated in formula *f*, then setting L true preserves satisfiability of *f*.

Therefore branch immediately on pure literals

DPLL *for real*

```
dpll(F, literal) {
        remove clauses containing literal
    shorten clauses containing literal
    if (F contains no clauses)
        return true;
    if (F contains empty clause)
        return false;
    if (F contains a unit or pure L)
        return dpll(F, L);
    choose P in F;
    if (dpll(F, P))
        return true;
    else
        return dpll(F, P); }
```

Where could we use a heuristic to improve performance?

Heuristic Search in DPLL

Heuristics are used in DPLL to select a proposition (non-unit, non-pure) for branching.

Idea: identify the most constrained variable because it is likely to create many unit clauses (simplify clauses).

MOMS heuristic says we should choose the propositional variables with the Maximum Occurrences in clauses of Minimal Size.

- Many more possible heuristics, e.g. alphabetical order of variables

DPLL *Examples*

$$(Q \vee \neg R \vee \neg P) \wedge (\neg R \vee P \vee Q) \wedge (R \vee Q) \wedge (\neg Q \vee P)$$

Variation 1

Order of variable assignments: Alphabetic

T/F assignment first: False

Variation 2

Order of variable assignments: MOMs heuristic

T/F assignment first: True

Variation 3

Order of variable assignments: MOMs heuristic

T/F assignment first: according to most occurrences

29

Variation 1. First branch on P with value False, ie. v(P)=0.
Then the clause set is transformed into:$(\neg R \vee Q) \wedge (R \vee Q) \wedge (\neg Q)$
Then we use a Unit clause $\neg Q$, so we set v(Q)=0.
Then the clause set becomes: $(\neg R) \wedge (R)$
Now both R and $\neg R$ are unit clauses, so choosing either next, e.g. v(R) = 1.
Then $\neg R$ is deleted, resulting in an empty clause, so the assignment returns 0, empty clause.
There is no need to backtrack on v(R)=1 as it is a unit-clause.
There is also no need to backtrack on Q, as it is a unit-clause

So we need to backtrack on P with the new value v(P)=1
This results in a new set of clauses $(Q \vee \neg R) \wedge (R \vee Q)$

Now Q is a pure literal and can be set to v(Q)=1.
Then both clauses can be deleted, and there are no clauses left.
So this is a solution, and we found a model v(P)=1 and v(Q)=1 and any valuation for R.

Variant 2: now we start with Q, as the variable occurring most often in the two smallest clauses (size 2).

We start with v(Q)=1.
This means clauses 1,2 and 3 are deleted, and we are left with P only.
This immediately gives us a model v(P)=1 and v(Q)=1 as before.

Variant 3: same as variant 2 as Q occurs more often positively (i.e. without negation) than negatively (with negation)

DPLL Summary

DPLL determines satisfiability of formulas in CNF, i.e.

DPLL(KB) = True *iff* KB is satisfiable

The worst case complexity of DP algorithm ***O(1.618^n).***

This is an improvement! Notice that

2^{50} = 1,125,899,906,842,624

1.618^{50} = 28,114,208,661

Part 3: Entailment by Refutation with DPLL

Calculating Entailment with DPLL by Refutation

A sentence a is entailed by KB K iff a is true for all models of K. KB $\models a$

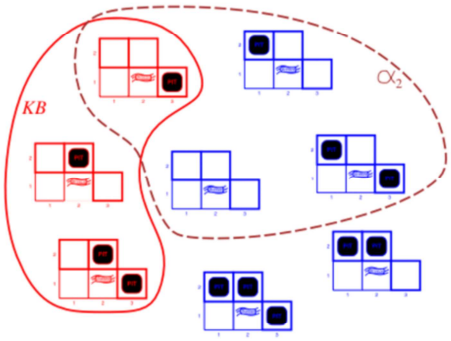
Therefore, to show that KB $\models a$, we must show there is no model for KB $\cup \neg a$.

This last step is a refutation because we are looking at the negation of a, $\neg a$.

We define a procedure KB $\vdash a$ {

1. Translate KB into CNF
2. Translate $\neg a$ into CNF
3. Add CNF(¬a) to CNF(KB)
4. Apply SAT solver (DPLL) on CNF(KB) $\cup$ CNF($\neg a$) until either
   a. Model is found (satisfisable)
   b. Search exhausted (unsatisfiable)
5. If satisfiable, return “NOT ENTAILED”, otherwise return “ENTAILED”

Wumpus World *take home message*



KB = wumpus-world rules ∧ observations

α_2 = "[2,2] is safe", $KB \not\models \alpha_2$

Given a symbolic declarative KB with the rules of the Wumpus game and the observations modeled in PL, we can use the DPLL calculus to inform the Agent by proving that [2,2] is a safe space (no Pits or Wumpus)

Semantics vs. Calculus

KB $\models \alpha$:

A sentence α is entailed by a KB K iff α is true in all models in KB.

KB $\vdash \alpha$:

DPLL of (CNF(KB) $\cup$ CNF($\neg\alpha$)) returns false

The intention with entailment with DPLL by refutation is that these two deliver the same result.

Properties of DPLL-Based Entailment

KB $\vdash a$ is sound; *only* the intended, correct formulas are proven.

$$\text{KB} \vdash a \text{ implies KB } \models a$$

KB $\vdash a$ is complete; *all* intended, correct formulas are proven.

$$\text{KB } \models a \text{ implies KB} \vdash a$$

KB $\vdash a$ is terminates; the procedure is finite.

KB $\vdash a$ is a decision procedure for entailment of propositional logic.

Intelligent Systems Framework

Efficient, simple, correct, well-understood, generic	Find best decision based on experience.	Weeks 6 & 7
	Find best decision with knowledge & reason.	Weeks 3, 4, 5
	Find best decision in multiple agent setting.	Weeks 1 & 2
	Find best decision with heuristics.	
	Systematically find all possible solutions.	

Progress in the intelligent systems framework.

Summary

Take Home Message

A **model** of a **KB** is an assignment of its propositions that satisfies the KB.

Rewriting propositional logic statement into **CNF** allows us to use **DPLL** to find models of a KB.

We can also use DPLL with **refutation** to assess whether an assignment is a model.

Thank you for your attention and have a lovely day.